

CEng 111 – Fall 2026
Week 2a

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Where does the word ‘algorithm’ come from?

Previously on CENG111!

- From a Persian mathematician, astronomer and geographer: Mohammed ibn-Musa al-Khwarizmi
- “Algorithmi” is the latin form of his name
- He contributed to science by
  - Decimal positional number system

    (e.g., $32 = 10^1x3 + 10^0x2$)
  - Presented the first systematic solutions to linear and quadratic equations
- In fact, the word “Algebra” comes from one of his operators (al-jabr: subtracting a number from both sides of an equation) for solving equations



Mohammed ibn-Musa al-Khwarizmi (780-850)

Source: Wikipedia

Previously on CENG111!

A formal definition of algorithm

- "Starting from an initial state and initial input (perhaps empty), the instructions describe a computation that, when executed, will proceed through a finite number of well-defined successive states, eventually producing "output" and terminating at a final ending state."



Previously on CENG111!

What is an algorithm?

- An algorithm is a list that looks like
  - STEP 1: Do something
  - STEP 2: Do something
  - STEP 3: Do something
  - . .
  - . .
  - . .
  - STEP N: Stop, you are finished

From "Invitation to Computer Science"

Previously on CENG111!

Valid Operations in Algorithms

- **Sequential** – simple well-defined task, usually declarative sentence.
- **Conditional-** "ask a question and select the next operation on the basis of the answer to the question – usually an "if-then" or "if then else"
- **Iterative-** "looping" instructions – repeat a set of instructions

From "Invitation to Computer Science"

Previously on CENG111!

How to represent algorithms

- Pseudo-code

Algorithm for Adding Two *m*-Digit Numbers

Given: $m \geq 1$ and two positive numbers each containing m digits, $a_{m-1}\, a_{m-2,} \ldots a_0$ and $b_{m-1}\, b_{m-2,} \ldots b_0$
Wanted: $c_m c_{m-1}\, c_{m-2} \ldots c_0$, where $c_m c_{m-1}\, c_{m-2} \ldots c_0 = (a_{m-1}\, a_{m-2} \ldots a_0) + (b_{m-1}\, b_{m-2} \ldots b_0)$

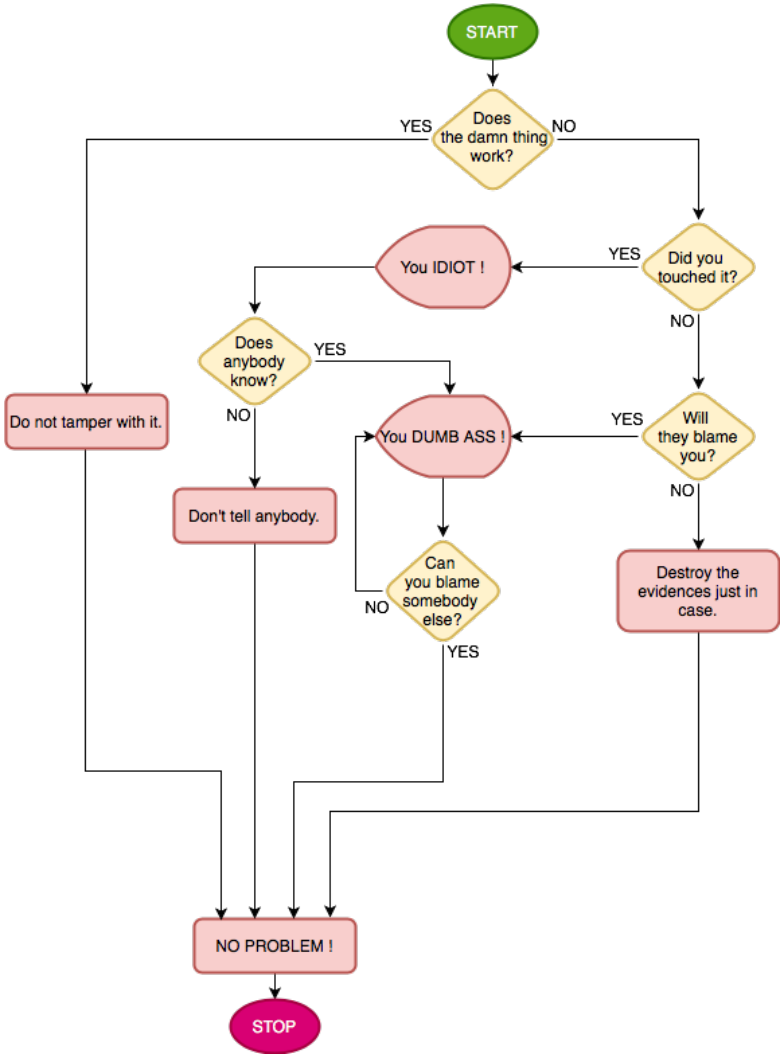
Algorithm:

Step 1 Set the value of *carry* to 0.
Step 2 Set the value of *i* to 0.
Step 3 While the value of *i* is less than or equal to $m - 1$, repeat the instructions in steps 4 through 6.
Step 4 Add the two digits a_i and b_i to the current value of *carry* to get c_i.
Step 5 If $c_i \geq 10$, then reset c_i to $(c_i - 10)$ and reset the value of *carry* to 1; otherwise, set the new value of *carry* to 0.
Step 6 Add 1 to *i,* effectively moving one column to the left.
Step 7 Set c_m to the value of *carry*.
Step 8 Print out the final answer, $c_m\, c_{m-1}\, c_{m-2} \ldots c_0$.
Step 9 Stop.

Previously on CENG111!

How to represent algorithms

- Flow-charts



Previously on CENG111!

"Computation"

- Digital vs. analog computation
- Sequential vs. parallel computation
- Batch vs. interactive computation
- Evolutionary, molecular, quantum computation
- "Physical computation" / "Digital Physics"
  - 'The whole universe is itself a computation'

Previously on CENG111!

Computation in our brain (cont'd)

- Each neuron gets input and produces an output using an "activation function"

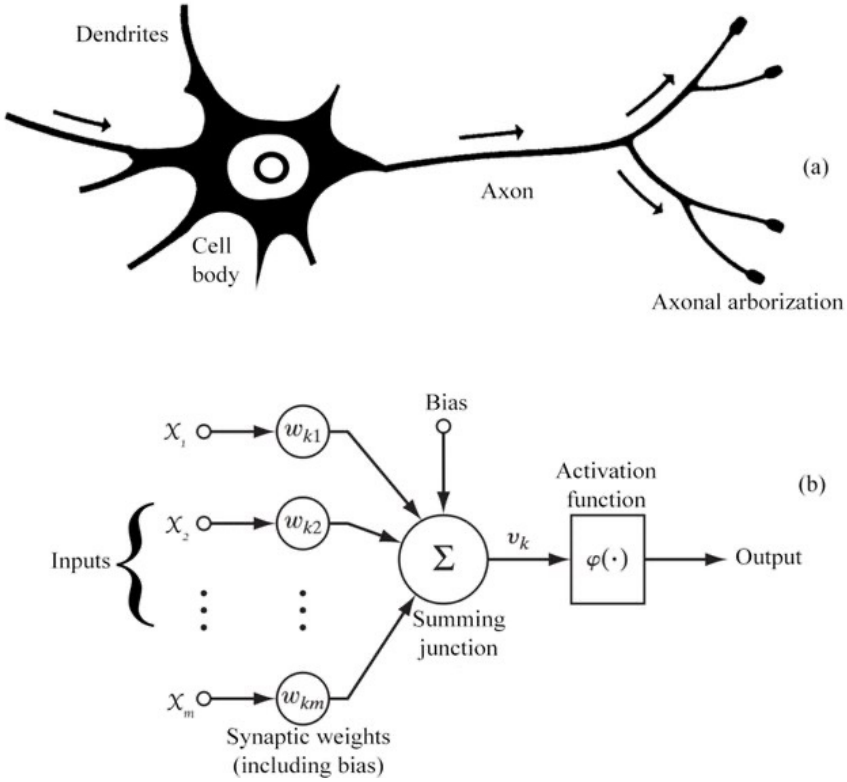
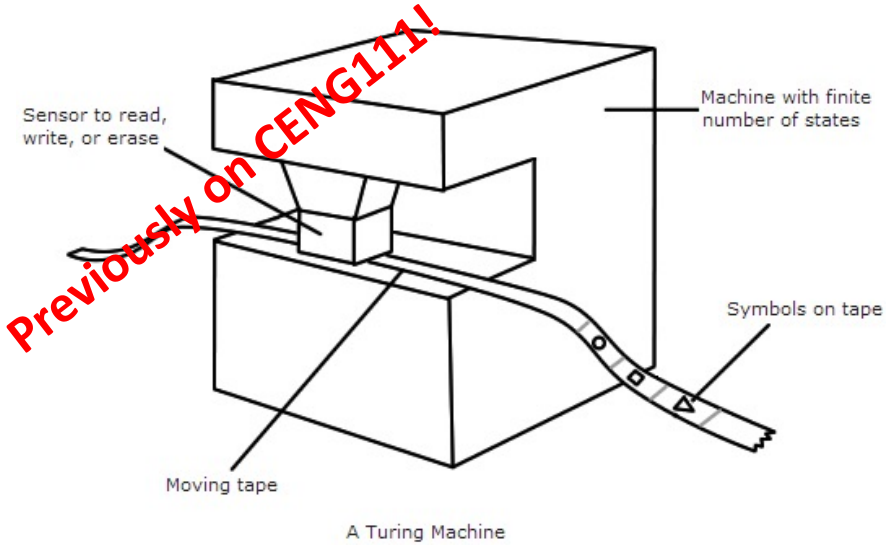
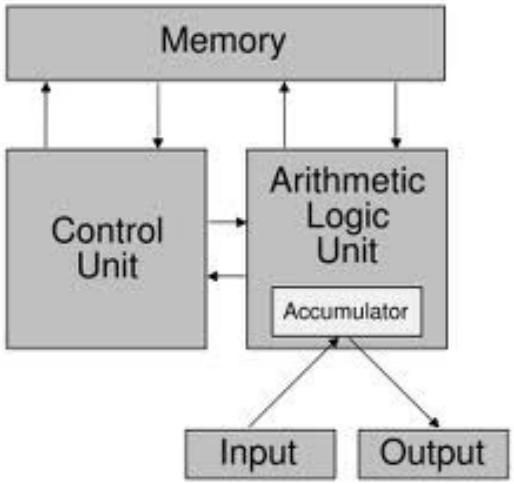


Fig: https://www.researchgate.net/publication/326417061_Modeling_Course_Achievements_of_Elementary_Education_Teacher_Candidates_with_Artificial_Neural_Networks/figures?lo=1



Turing Machine



Von Neumann Architecture

DIGITAL COMPUTATION



Today

- Digital Computation:
  - Boolean Logic
  - Computation Components
  - Von Neumann Architecture

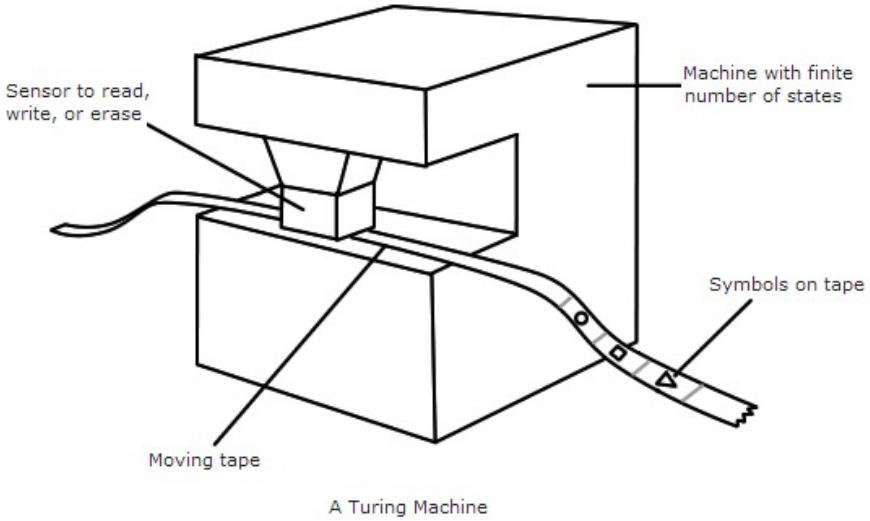
Administrative Notes

- History of computing quiz
  - Material shared
  - Online quiz:
    - Monday (12 Oct) between 19:00 and 19:30
    - On ODTUClass page of the course.
- Submitting busy hours
  - Deadline: 9 Oct 23:55
- Linux Workshops:
  - 7 Oct 18:00 & 13 Oct 18:30

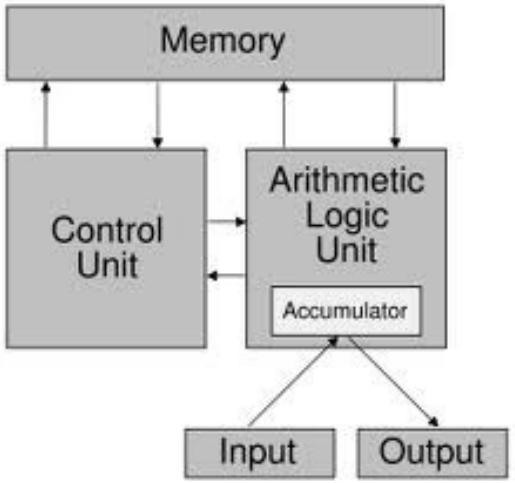
Motivation Note



https://open.spotify.com/episode/17eW18fUlqiHjh1FFz00N0?si=155ed5d3404e450a



Turing Machine



Von Neumann
Architecture

DIGITAL COMPUTATION

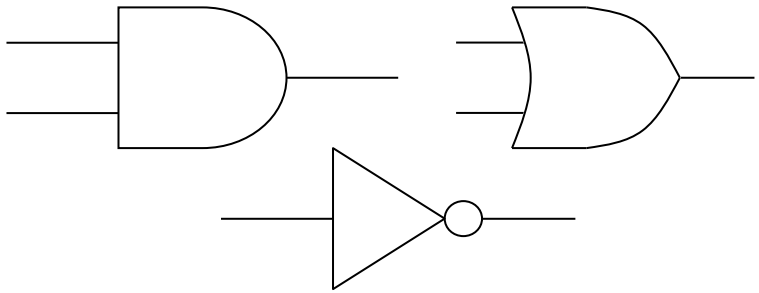
A computer



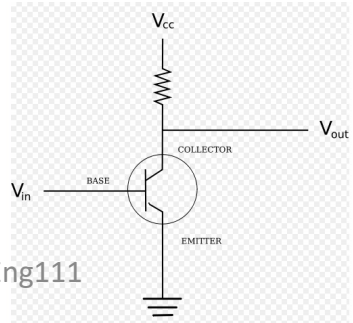




Devices

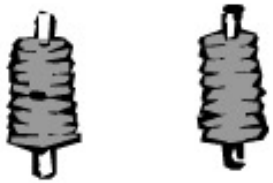
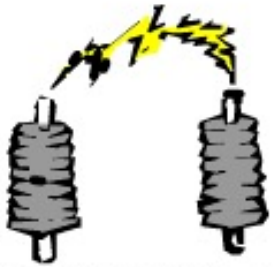


Gates

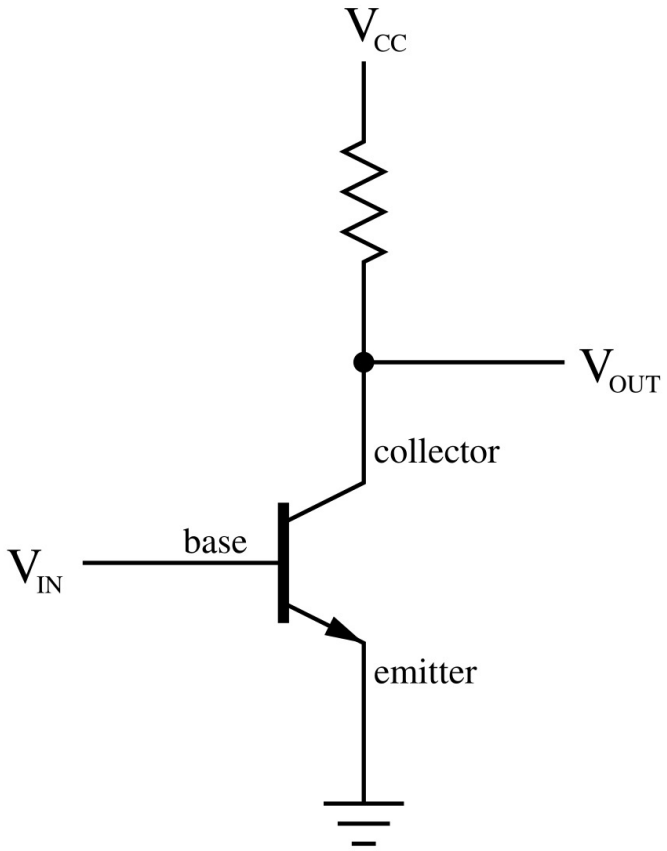


Transistors

Everything in a PC is Binary
… well, almost …

States of a Bit			
0	 2+2=5 FALSE	 OFF	 LOW VOLTAGE
1	 2+2=4 TRUE	 ON	 HIGH VOLTAGE

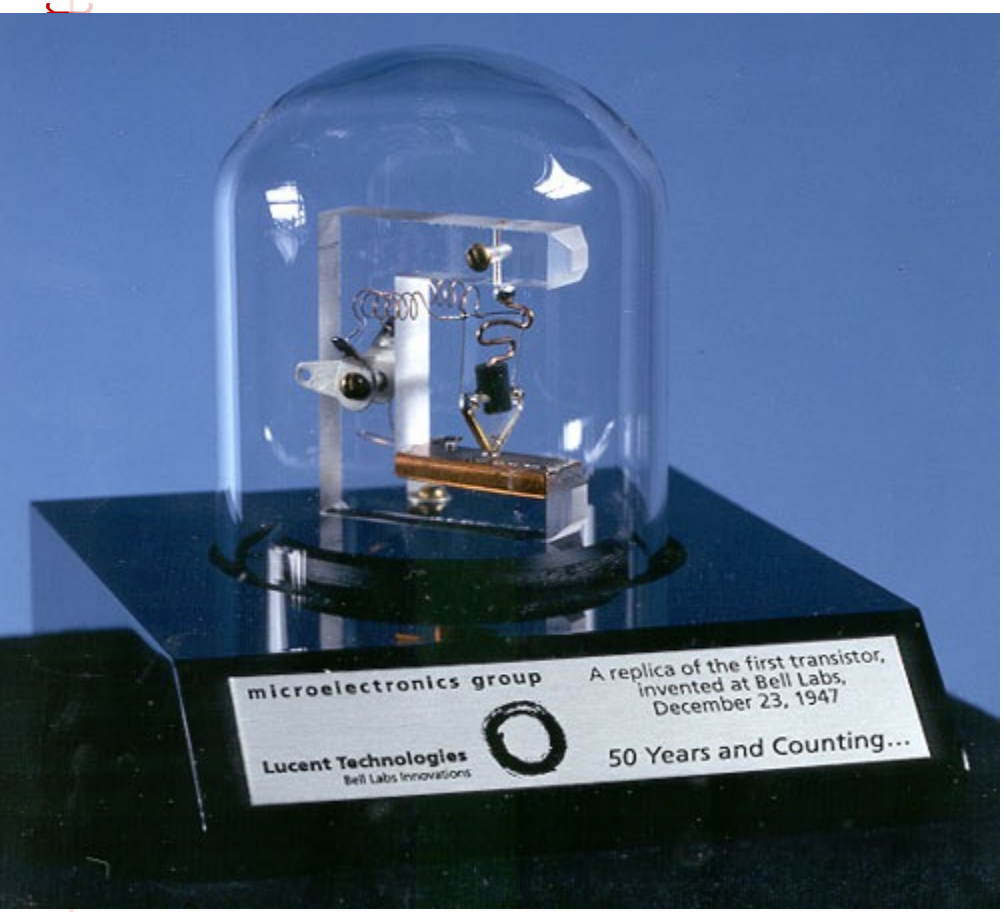
A transistor



This circuit functions as a switch. In other words, based on the *control* voltage, the circuit either passes Vin to output or not.

Fig: https://en.wikipedia.org/wiki/Transistor

Examples of transistors



Replica of the first transistor



A set of transistors, depicting the fast change in technology.

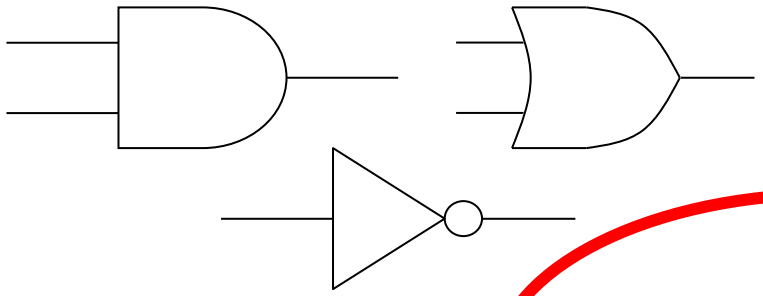
A computer



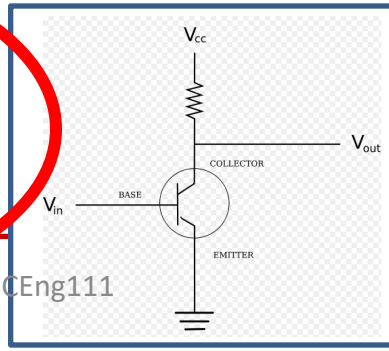




Devices



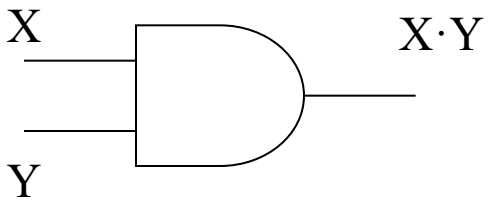
Gates



Transistors

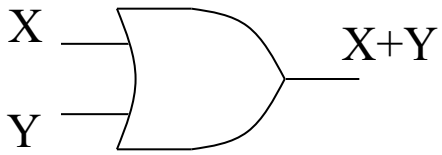
AND gate

X	Y	X·Y
0	0	0
0	1	0
1	0	0
1	1	1



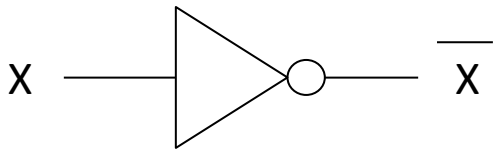
OR Gate

X	Y	X+Y
0	0	0
0	1	1
1	0	1
1	1	1



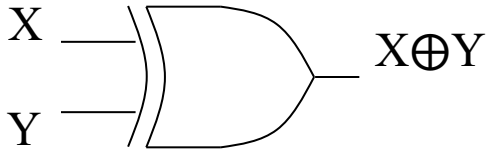
NOT Gate

X	$\overline{X}$
0	1
1	0



XOR Gate

X	Y	X⊕Y
0	0	0
0	1	1
1	0	1
1	1	0



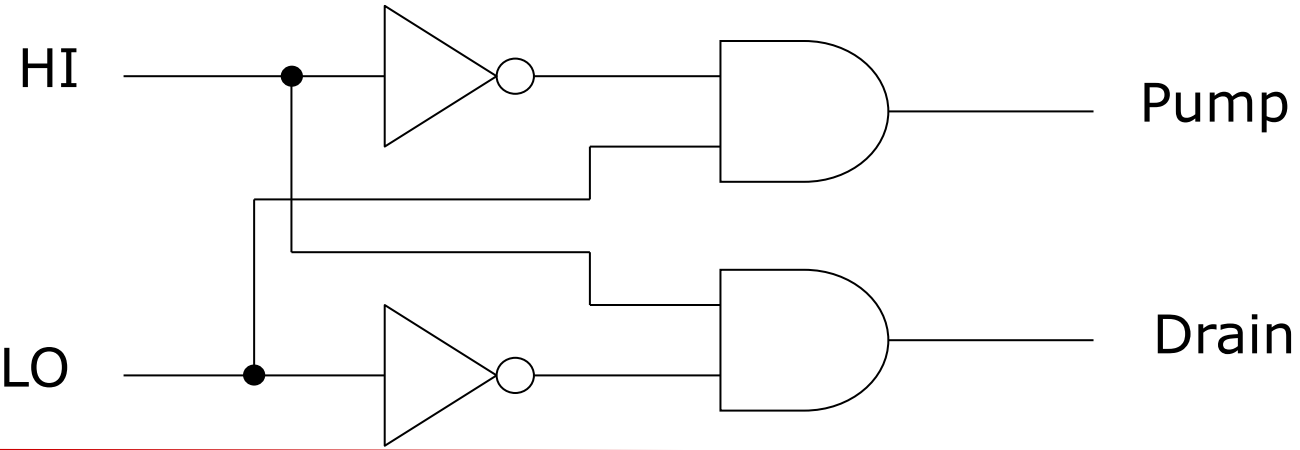
An example problem: Water Tank

METU Computer Engineering

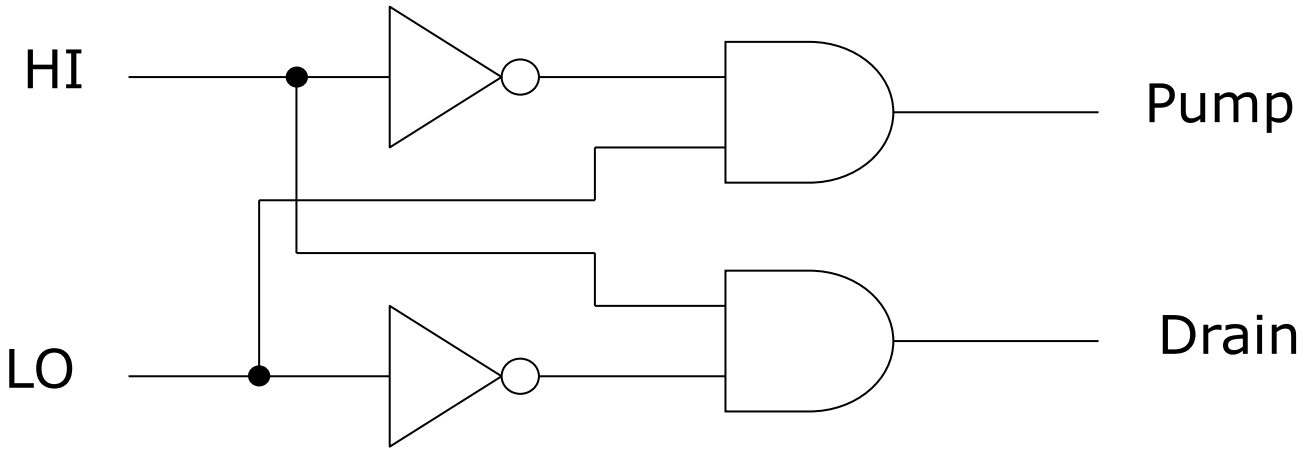
Truth Table Representation

HI	LO	Pump	Drain	
0	0	0	0	→ Tank level is OK
0	1	**1**	0	→ Low level, pump more in
1	0	0	**1**	→ High level, drain some out
1	1	x	x	→ Inputs cannot occur

Schematic Representation



Boolean Logic/Algebra



Pump = HI'.LO
Drain = HI.LO'

Boolean formula describing the circuit.

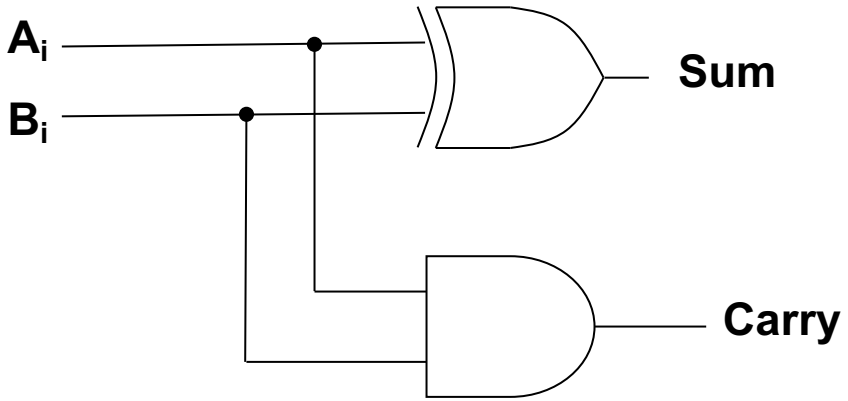
The binary addition

$$\begin{array}{r} 0 \\ +\,0 \\ \hline 0 \end{array} \qquad \begin{array}{r} 1 \\ +\,0 \\ \hline 1 \end{array} \qquad \begin{array}{r} 0 \\ +\,1 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\,1 \\ \hline 10 \end{array}$$

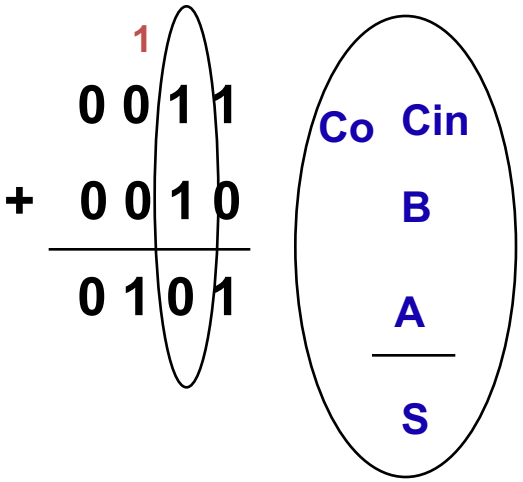
Question (Binary notation) : 111010 + 11011 = ?

1-bit Half-adder

A_i	B_i	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



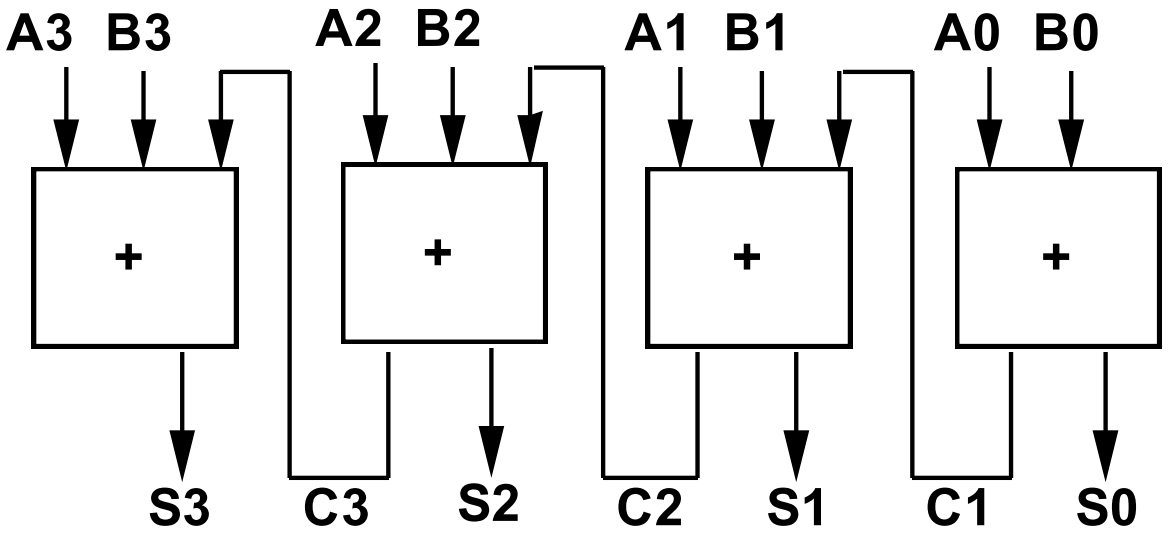
1-bit full-adder



A	B	CI	S	CO
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

METU Computer Engineering

N-bit Adder



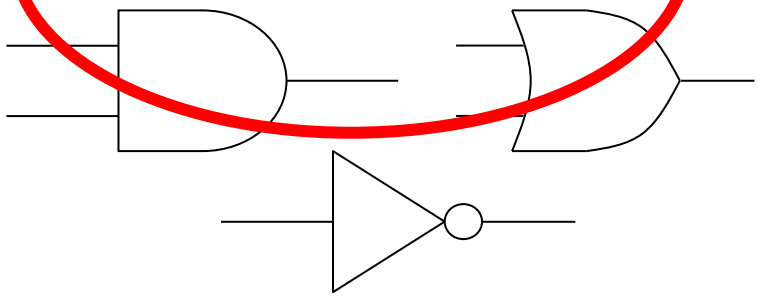
A computer



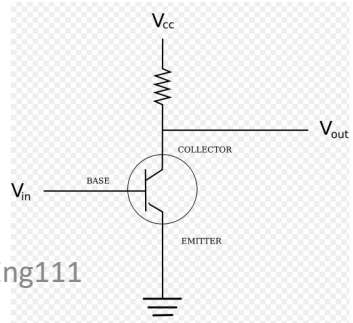




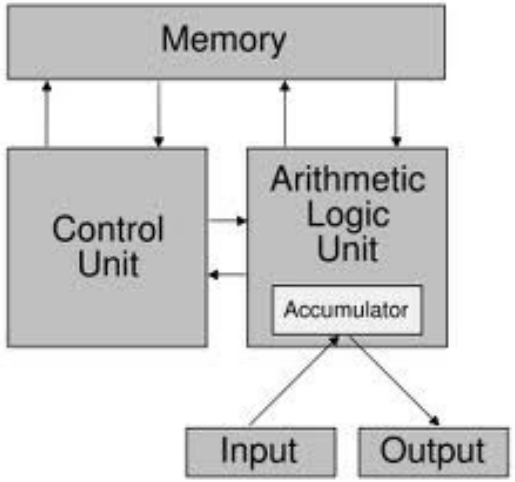
Devices



Gates



Transistors



Computer Organization

VON NEUMANN ARCHITECTURE & ITS IMPLEMENTATION

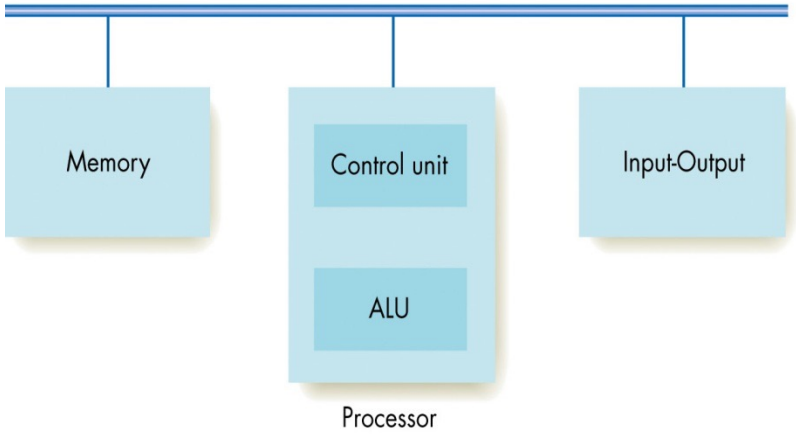


Von Neumann Architecture & Its Implementation

- How are instructions coded?
- How are instructions executed?
- How do the different subcomponents interact?
  - Memory
  - ALU
  - The Bus System
  - Registers

The Components of a Computer System

- Von Neumann architecture has four functional units:
  - Memory
  - Input/Output
  - Arithmetic/Logic unit
  - Control unit



- Sequential execution of instructions
- Stored program concept

METU Computer Engineering

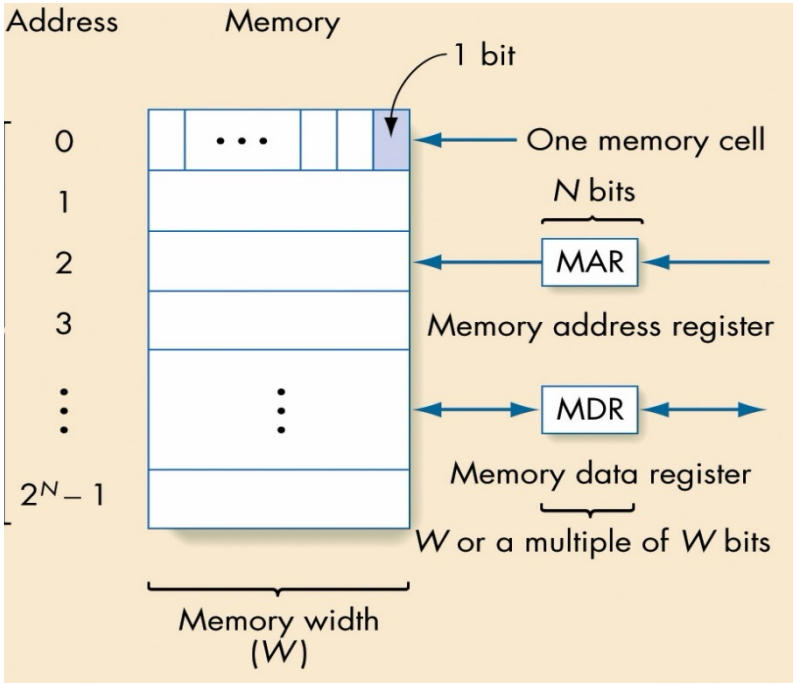


Memory and Cache

- Information is stored and fetched from memory subsystem
- Memory maps addresses to memory locations
- Cache memory keeps values currently in use in faster memory to speed access times

Memory and Cache (continued)

- RAM (Random Access Memory)
  Often called *memory*, *primary memory*
  - Memory made of addressable "cells"
  - Cell size is 8 bits
    - Nowadays, it is 32 or 64 bits.
  - All memory cells accessed in equal time
  - Memory address
    - Unsigned binary number with N bits
    - Address space is then 2^N cells



Memory Unit	Description
Bit	Binary Digit 1 or 0
Byte	8 bits
KiloByte (KB)	1024 Bytes
MegaByte (MB)	1024 KB
GigaByte (GB)	1024 MB
TeraByte (TB)	1024 GB
PetaByte (PB)	1024 TB
HexaByte or exaByte (EB)	1024 PB
ZettaByte (ZB)	1024 EB
YottaByte (YB)	1024 ZB
BrontoByte	1024 YB
GeopByte	1024 Bronto Bytes

Table: https://www.krivalar.com/computer-memory-units



Memory and Cache (continued)

- Rapid access, low capacity “warehouse”
- Retains information entered through input unit
- Retains info that has already been processed until can be sent to output unit

Memory and Cache (continued)

- Parts of the memory subsystem
  - Fetch/store (or Read/Write) controller
    - Fetch: retrieve a value from memory
    - Store: store a value into memory
  - Memory address register (MAR)
  - Memory data register (MDR)

METU Computer Engineering



Memory and Cache (continued)

- Fetch operation
  - The address of the desired memory cell is moved into the MAR
  - Fetch/store controller signals a “fetch,” accessing the memory cell
  - The value at the MAR’s location flows into the MDR



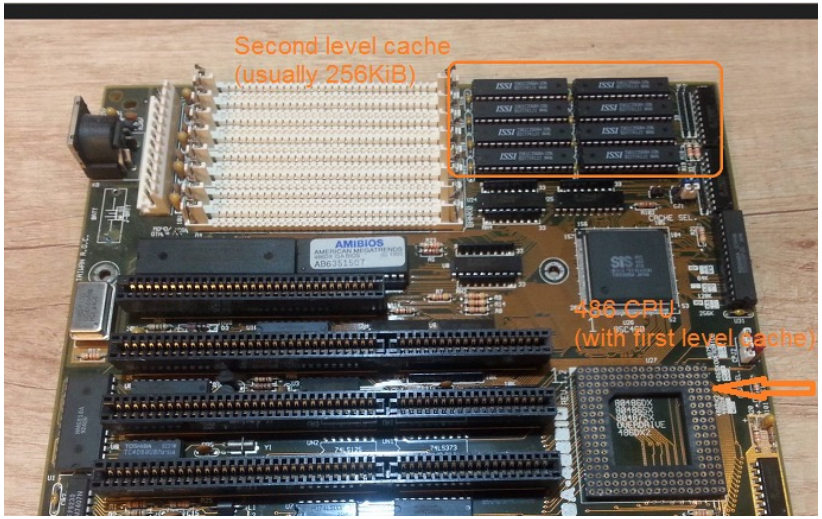
Memory and Cache (continued)

- Store operation
  - The address of the cell where the value should go is placed in the MAR
  - The new value is placed in the MDR
  - Fetch/store controller signals a “store,” copying the MDR’s value into the desired cell



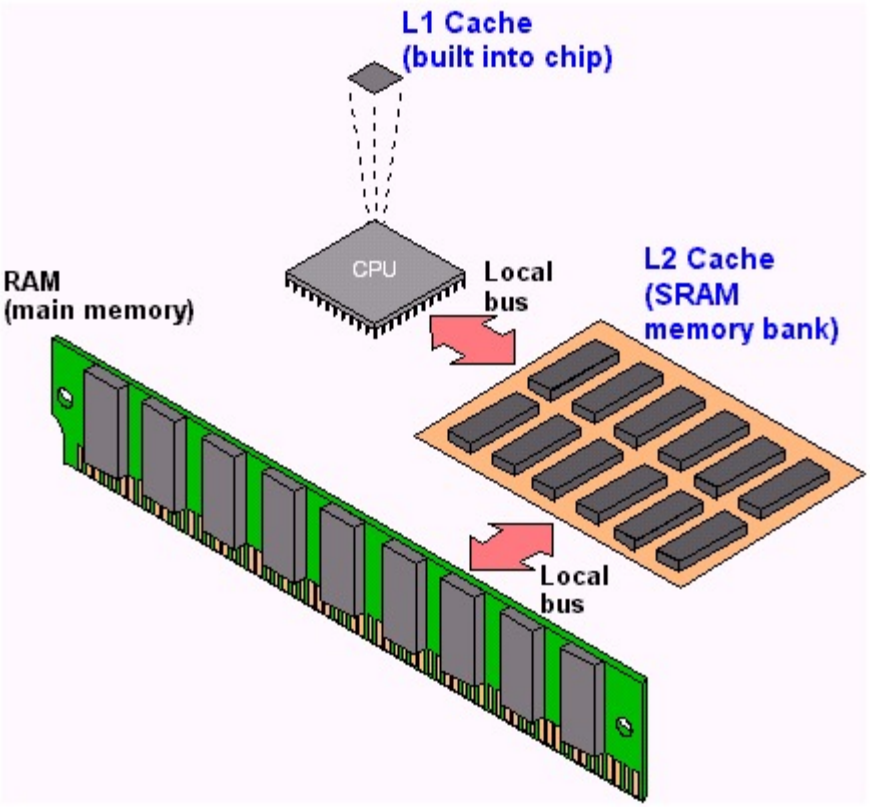
Cache Memory

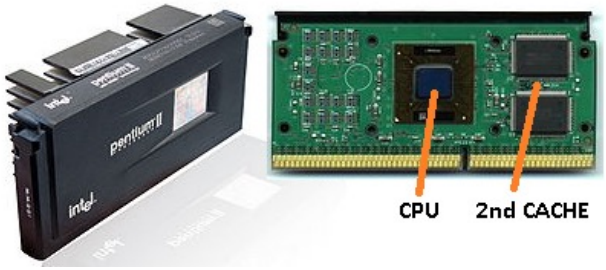
- Memory access is much slower than processing time
- Faster memory is too expensive to use for all memory cells
- Locality principle
  - Once a value is used, it is likely to be used again
- Small size, fast memory just for values currently in use speeds computing time

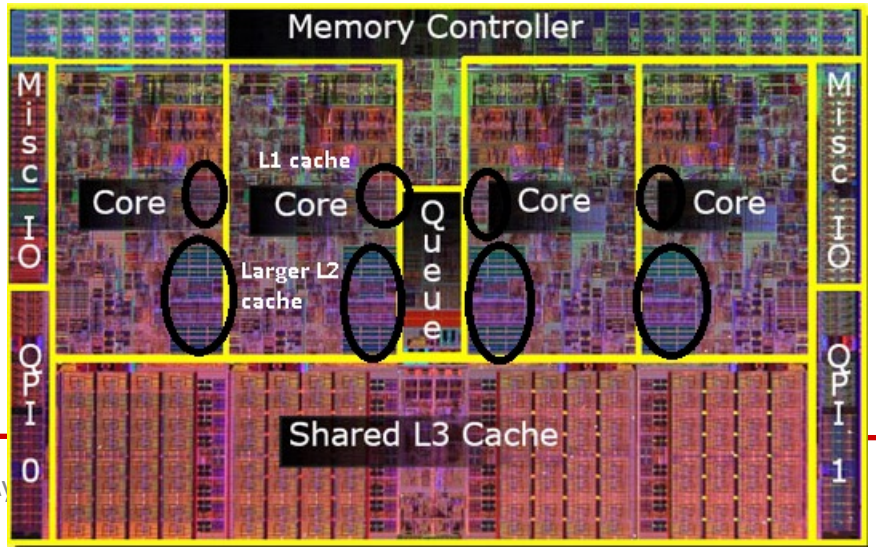


80486: (1989)

From Computer Desktop Encyclopedia
© 1999 The Computer Language Co. Inc.

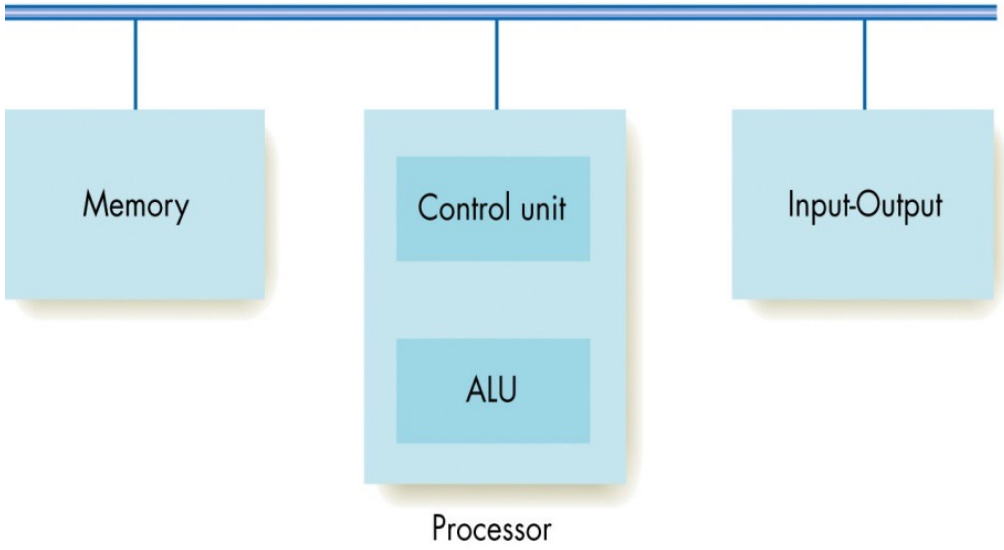






Central Processing Unit (CPU)

- Control Unit
- Arithmetic Logic Unit (ALU)



METU Computer Engineering

The Control Unit



- Manages stored program execution
- Task

1. Fetch from memory the next instruction to be executed
2. Decode it: determine what is to be done
3. Execute it: issue appropriate command to ALU, memory, and I/O controllers



Machine Language Instructions

- Can be decoded and executed by control unit
- Parts of instructions
  - Operation code (op code)
    - Unique unsigned-integer code assigned to each machine language operation
  - Address field(s)
    - Memory addresses of the values on which operation will work



Typical Machine Language Instruction Format

More on Instructions



- LOAD X -> Load register R with the contents of memory cell X
- STORE X -> Store register R into memory cell X
- MOVE X, Y -> Copy the contents of X into Y
- ADD X -> Add contents of X to the contents of R
- ADD X,Y -> Add contents of X to the contents of Y, and put the result in register R
- COMPARE X, Y-> Set GT (greater than), EQ (equal) and LT (less than) condition codes
- JUMP X-> Jump unconditionally to the instruction in cell X
- JUMPGT X-> Jump, if GT=1, to the instruction in cell X
- HALT



Machine Language Instructions (continued)

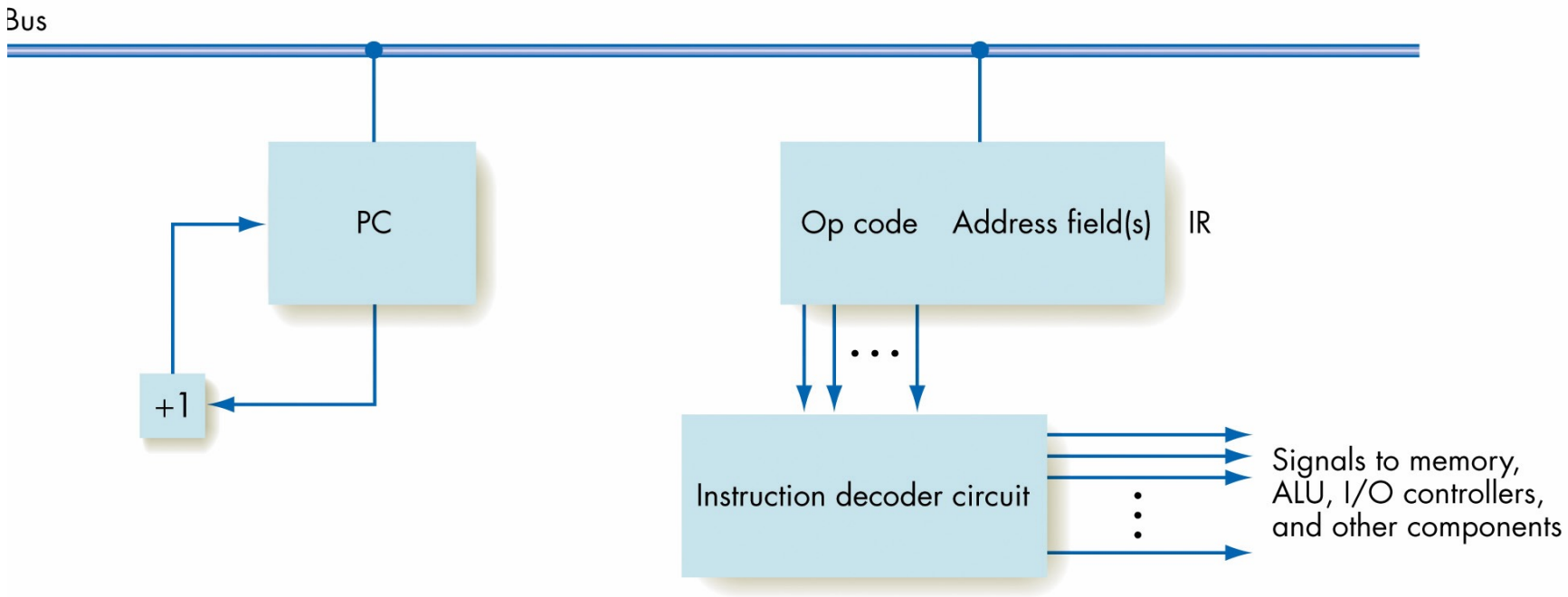
- Types of machine instructions:
  - Data transfer
    - Move values to and from memory and registers
  - Arithmetic/logic
    - Perform ALU operations that produce numeric values
  - Compares
    - Set bits of compare register to hold result
  - Branches
    - Jump to a new memory address to continue processing



Control Unit Registers and Circuits

METU Computer Engineering

- Parts of control unit
  - Linked to other subsystems
  - Instruction decoder circuit
  - Two special registers:
    - Program Counter (PC)
      - Stores the memory address of the next instruction to be executed
    - Instruction Register (IR)
      - Stores the code for the current instruction



Organization of the Control Unit Registers and Circuits